HTML Tables, Forms & Multimedia

Tools Used in This Module

Visual Studio Code (VSCode) — write and preview all HTML files locally. Install the Live Server extension for instant in-browser previews.

CodePen (codepen.io) — used especially for Multimedia tasks. CodePen lets you test embed codes (YouTube, Spotify, etc.) and external media URLs without needing a local server or file hosting.

Open VSCode for Tables and Forms. Switch to CodePen for Multimedia experiments.

01 HTML Tables

HTML Tables

1.1 What Is an HTML Table?

An HTML table organises data into rows and columns — exactly like a spreadsheet. Tables are the right tool whenever your content is truly tabular: timetables, price lists, results, comparison charts, schedules.

Tables are NOT for layout

Never use <table> to position elements on a page. That was a 1990s habit. Modern layouts use CSS Flexbox or CSS Grid. Use tables only for actual data.

Setting Up in VSCode

1. Open VSCode and create a new file: File → New File → Save As table-practice.html
2. Type ! and press Tab to generate the HTML5 boilerplate (Emmet shortcut).
3. Write your table code inside <body>.
4. Right-click the file → Open with Live Server to see your table in the browser instantly.

1.2 Core Table Elements

Every table is built from four core elements:

Element	Full Name	Purpose
`<table>`	Table	The outer container — wraps everything
`<tr>`	Table Row	Defines one horizontal row
`<th>`	Table Header	Header cell — bold and centred by default
`<td>`	Table Data	Standard data cell — holds content

A minimal table example:

```
table-practice.html
<!-- table-practice.html — type this in VSCode -->
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>City</th>
  </tr>
  <tr>
    <td>Amara</td>
    <td>22</td>
    <td>Lagos</td>
  </tr>
  <tr>
    <td>Chidi</td>
    <td>25</td>
    <td>Abuja</td>
  </tr>
</table>
```

> **VSCode Tip**
> Select any HTML tag in VSCode and press Ctrl+D (Windows) or Cmd+D (Mac) to select the next matching tag. This makes editing repeated elements like <td> much faster.

1.3 Semantic Table Sections

For real-world tables, wrap rows in semantic section elements. These help browsers, screen readers, and CSS stylesheets understand the table structure:

- **`<thead>`** — groups the header row(s) at the top
- **`<tbody>`** — groups the main data rows
- **`<tfoot>`** — groups summary or total rows at the bottom

```
table-practice.html
```

```html
<table>
  <caption>Monthly Sales — March 2026</caption>
  <thead>
    <tr>
      <th scope="col">Product</th>
      <th scope="col">Price</th>
      <th scope="col">Qty</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Rice (5kg)</td>
      <td>&#8358;3,500</td>
      <td>10</td>
    </tr>
    <tr>
      <td>Beans (2kg)</td>
      <td>&#8358;1,800</td>
      <td>5</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Total</td>
      <td></td>
      <td>15</td>
    </tr>
  </tfoot>
</table>
```

1.4 Spanning Cells

Use `colspan` to stretch a cell across multiple columns, and `rowspan` to stretch it across multiple rows:

```html
table-practice.html
<table>
  <tr>
    <!-- This header spans all 3 columns -->
    <th colspan="3">Student Results — Term 1</th>
  </tr>
  <tr>
    <!-- This cell spans 2 rows (Maths + English) -->
    <td rowspan="2">Chinyere</td>
    <td>Maths</td>
    <td>87%</td>
  </tr>
  <tr>
    <!-- No first <td> here — rowspan above covers it -->
    <td>English</td>
    <td>91%</td>
```

```
    </tr>
  </table>
```

> **Common colspan/rowspan Mistake**
> When you use rowspan='2' on a cell, the NEXT row must have one fewer <td>. Forgetting this shifts all cells out of alignment.

1.5 Table Attributes Quick Reference

Attribute / Element	Example	What It Does
`<caption>`	<caption>Title</caption>	Visible title above the table — improves accessibility
`<thead>`	wraps header rows	Groups header rows; allows independent CSS styling
`<tbody>`	wraps data rows	Groups main data rows
`<tfoot>`	wraps footer rows	Groups footer/summary rows
`scope`	scope="col"	On <th>: tells screen readers which column/row it heads
`colspan`	colspan="2"	Merges cell across 2 columns
`rowspan`	rowspan="3"	Merges cell down 3 rows

1.6 Accessibility in Tables

1. **Use <th> for all headers**, never <td>
2. Add `scope="col"` or `scope="row"` to every <th>
3. Always include `<caption>` to describe the table
4. Never use tables for page layout

1.7 Practice Task — Tables

> **Task: Class Timetable**
> In VSCode, create a file called timetable.html.
> Build an HTML table showing a Monday–Friday class timetable. Requirements:
> • At least 5 subjects across the week
> • Use <thead>, <tbody>, and a <caption>
> • Use colspan to merge any cell where a subject runs for two periods

- Use scope='col' on all header cells

Preview with Live Server and check your layout in the browser.

02 HTML Forms

HTML Forms

2.1 What Is an HTML Form?

A form collects input from the user and sends it somewhere for processing. Every login page, sign-up form, search box, and contact page is built with HTML forms. The browser handles the UI — you write the structure in HTML.

Setting Up in VSCode

1. Create a new file: form-practice.html
2. Use the ! + Tab Emmet shortcut to scaffold the HTML5 boilerplate.
3. All form code goes inside <body>.
4. Open with Live Server to test your form inputs in real time.

Note: For now, your form won't actually submit data — that requires a server-side language (covered in a later module). Focus on building the structure correctly.

2.2 The <form> Element

The `<form>` element is the container for all form controls. Its two key attributes control how and where the data is sent:

```
form-practice.html
<!-- action: WHERE to send the data -->
<!-- method: HOW to send it (GET or POST) -->
<form action="/submit" method="POST">
  <!-- all inputs go inside here -->
</form>
```

Attribute	Values	What It Does
`action`	"/submit" "/register"	URL that receives the form data when submitted

Attribute	Values	What It Does
`method`	"GET" "POST"	GET: data appears in the URL. POST: data is hidden in the request body — use for passwords and sensitive info

2.3 The <input> Element

The `<input>` element is the most common form control. It is self-closing (no closing tag). The `type` attribute determines what kind of input is rendered:

type value	How It Looks / Behaves	When to Use
`type="text"`	Single-line text box	Names, usernames, any short text
`type="email"`	Text box + email format validation	Email addresses — browser checks for @ sign
`type="password"`	Text box that hides characters	Passwords, PINs, secret codes
`type="number"`	Numeric input with spin buttons	Age, quantity, scores
`type="tel"`	Text box (phone keyboard on mobile)	Phone numbers
`type="date"`	Date picker popup	Dates of birth, appointment dates
`type="checkbox"`	Tick box — on or off	Multi-select options, agree/disagree
`type="radio"`	Dot selector — one from a group	Single choice from named options
`type="file"`	Browse button + file path display	Uploading photos, documents
`type="submit"`	Clickable submit button	Submitting the form
`type="reset"`	Clears all inputs in the form	Reset / clear form button
`type="hidden"`	Invisible — not shown to user	Sending behind-the-scenes data (tokens)

2.4 Labels — Always Required

Every input must have a `<label>` connected to it. Labels are not optional — they are essential for accessibility (screen readers) and usability (clicking the label focuses the input). Connect them using matching `for` and `id` values:

```
form-practice.html
<!-- for='username' must match id='username' -->
```

```
<label for="username">Username:</label>
<input type="text"
       id="username"
       name="username"
       placeholder="e.g. amara_22"
       required>
```

> **name vs id**
> Both name and id are required but serve different purposes.
> id — links the input to its <label> (used in the browser)
> name — the key used when data is submitted to the server
> Always include both on every input.

2.5 Important Input Attributes

Attribute	Example	What It Does
`name`	name="email"	Key sent to the server with the form data — required
`id`	id="email"	Links the input to its <label> — required
`placeholder`	placeholder="..."	Ghost hint text shown before the user types
`value`	value="default text"	Sets a pre-filled default value
`required`	required	HTML5 — form will not submit if this field is empty
`disabled`	disabled	Greys out the field — user cannot interact with it
`readonly`	readonly	User can see the value but not change it
`min / max`	min="18" max="65"	For number/date inputs — restricts the allowed range
`minlength`	minlength="8"	Minimum number of characters required
`maxlength`	maxlength="50"	Maximum number of characters allowed
`autocomplete`	autocomplete="off"	Disables browser autofill — useful for sensitive fields

2.6 Checkboxes and Radio Buttons

Checkboxes — Multiple Selections

Checkboxes let the user select any number of options. Each checkbox is independent.

```
form-practice.html
<p>Select your interests:</p>
<label>
  <input type="checkbox" name="interest" value="html"> HTML
</label>
<label>
  <input type="checkbox" name="interest" value="css"> CSS
</label>
<label>
  <input type="checkbox" name="interest" value="js"> JavaScript
</label>
```

Radio Buttons — Single Selection

Radio buttons belong to a group defined by a shared `name` attribute. Only one option in the group can be selected at a time.

```
form-practice.html
<p>Gender:</p>
<label>
  <input type="radio" name="gender" value="male"> Male
</label>
<label>
  <input type="radio" name="gender" value="female"> Female
</label>
<label>
  <input type="radio" name="gender" value="other"> Prefer not to say
</label>
```

2.7 Textarea and Select

Textarea — Multi-line Text

Use `<textarea>` when users need to type longer text like messages or comments.

```
form-practice.html
<label for="message">Your Message:</label>
<textarea id="message"
          name="message"
          rows="5"
          cols="40"
          placeholder="Write your message here...">
</textarea>
```

Select Dropdown

The `<select>` element creates a dropdown. Each choice is an `<option>`. Use `<optgroup>` to group related options.

```
form-practice.html
<label for="course">Choose a course:</label>
<select id="course" name="course">
  <option value="">-- Select --</option>
  <optgroup label="Frontend">
    <option value="html">HTML Fundamentals</option>
    <option value="css">CSS Styling</option>
    <option value="js">JavaScript</option>
  </optgroup>
  <optgroup label="Backend">
    <option value="node">Node.js</option>
  </optgroup>
</select>
```

2.8 Fieldset, Legend & Button

Fieldset & Legend

Wrap related inputs in `<fieldset>` and use `<legend>` as its title. This draws a visible box around the group and helps screen readers understand the grouping.

```
form-practice.html
<fieldset>
  <legend>Personal Information</legend>
  <label for="fname">First Name:</label>
  <input type="text" id="fname" name="fname" required>
  <label for="lname">Last Name:</label>
  <input type="text" id="lname" name="lname" required>
</fieldset>
```

Button

The `<button>` element is more flexible than `input type="submit"` because you can put any content (text, icons) inside it.

```
form-practice.html
<!-- Submits the form -->
<button type="submit">Create Account</button>

<!-- Clears all inputs -->
<button type="reset">Clear Form</button>

<!-- Does nothing by default — use with JavaScript -->
<button type="button">Preview</button>
```

2.9 HTML5 Built-in Validation

HTML5 can validate user input automatically — no JavaScript needed. The browser shows its own error messages if validation fails:

```
form-practice.html
<!-- required: field must be filled -->
<input type="text" name="name" required>

<!-- email: checks for valid email format -->
<input type="email" name="email" required>

<!-- min/max: number must be between 18 and 65 -->
<input type="number" name="age" min="18" max="65">

<!-- minlength: password must be at least 8 characters -->
<input type="password" name="pass" minlength="8" required>

<!-- pattern: only letters, 3 to 20 characters -->
<input type="text" name="username" pattern="[A-Za-z]{3,20}" required>
```

2.10 Complete Form Example

```
form-practice.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Form</title>
</head>
<body>
  <h1>Create Your Account</h1>
  <form action="/register" method="POST">

    <fieldset>
      <legend>Personal Details</legend>
      <label for="fname">First Name:</label>
      <input type="text" id="fname" name="fname" required><br>

      <label for="dob">Date of Birth:</label>
      <input type="date" id="dob" name="dob"><br>

      <p>Gender:</p>
      <label><input type="radio" name="gender" value="m"> Male</label>
      <label><input type="radio" name="gender" value="f"> Female</label>
    </fieldset>

    <fieldset>
      <legend>Account Details</legend>
      <label for="email">Email:</label>
```

```
      <input type="email" id="email" name="email" required><br>

      <label for="pass">Password:</label>
      <input type="password" id="pass" name="pass" minlength="8"
required><br>

      <label for="course">Course:</label>
      <select id="course" name="course">
        <option value="html">HTML Fundamentals</option>
        <option value="css">CSS Styling</option>
      </select>
    </fieldset>

    <label>
      <input type="checkbox" name="agree" required>
      I agree to the Terms and Conditions
    </label><br>

    <button type="submit">Register</button>
    <button type="reset">Clear</button>
  </form>
</body>
</html>
```

2.11 Practice Task — Forms

Task: Student Enrolment Form

In VSCode, create a file called enrolment.html.

Build a Student Enrolment Form with these fields:

- Full name (text, required)
- Email address (email, required)
- Phone number (tel)
- Date of birth (date)
- Gender (radio: Male / Female / Other)
- Preferred course (select with at least 4 options using optgroup)
- Message / notes (textarea, 5 rows)
- Profile photo upload (file)
- Agree to terms (checkbox, required)
- Submit and Reset buttons

Group Personal Details and Account Details in separate <fieldset> elements.

Test all validation by trying to submit with empty required fields.

03 HTML Multimedia

HTML Multimedia

3.1 VSCode vs CodePen for Multimedia

Multimedia in HTML involves images, audio, video, and embeds. Depending on what you're working on, you'll use different tools:

Use VSCode for

- Local image files (<img> pointing to files you have on your computer)
- Audio and video files you have saved locally (.mp3, .mp4, etc.)
- Building the overall page structure

Limitation: Local audio/video files may not load in the browser directly from a file:// path due to security restrictions. Use Live Server in VSCode — it serves your files over http://localhost which removes this restriction.

Use CodePen for

- YouTube, Vimeo, Spotify, and other embeds (<iframe>)
- Testing images from public URLs (Unsplash, Pexels, your own hosted images)
- Audio from public URLs (SoundCloud embeds, CDN-hosted mp3 links)
- Quickly experimenting with media attributes (autoplay, loop, poster, etc.)

Why CodePen? It runs on a real web server so embeds and external URLs work perfectly without any setup.

3.2 Images — <img>

The `<img>` element embeds an image. It is void — no closing tag. The `src` and `alt` attributes are both required.

```
multimedia.html
<!-- Local file (VSCode + Live Server) -->
<img src="images/profile.jpg"
     alt="Profile photo of Amara Okafor"
     width="300"
     height="300"
     loading="lazy">

<!-- Public URL (works in both VSCode and CodePen) -->
<img src="https://images.unsplash.com/photo-example"
     alt="A scenic mountain landscape"
     width="600">
```

Attribute	Example	What It Does
`src`	src="photo.jpg"	Path or URL to the image — required
`alt`	alt="Description"	Alternative text — required for accessibility and SEO
`width`	width="400"	Display width in pixels
`height`	height="300"	Display height in pixels
`loading`	loading="lazy"	Defers loading until image is near the viewport — improves page speed
`title`	title="Tooltip"	Shows a tooltip on hover

Supported Image Formats

- **JPEG / JPG** — best for photos; great compression; no transparency
- **PNG** — best for graphics, logos, screenshots; supports transparency
- **GIF** — supports simple animations; limited to 256 colours
- **SVG** — scalable vector; perfect for icons and logos; resolution-independent
- **WebP** — modern format; best quality-to-size ratio; recommended for new projects

Responsive Images — Always Add This CSS

```
styles
/* Put this in a <style> tag or CSS file */
img {
  max-width: 100%;
  height: auto;
}
```

Never skip the alt attribute

An image with no alt text is invisible to screen readers and bad for SEO.

For meaningful images: alt='A profile photo of Amara Okafor'

For decorative images only: alt='' (empty string — tells screen readers to skip it)

3.3 Figure and Figcaption

Wrap images (and other media) in `<figure>` and add a visible caption with `<figcaption>`. This is the semantic, accessible way to associate a caption with media.

```
multimedia.html
<figure>
  <img src="html-diagram.png"
       alt="Diagram showing the structure of an HTML document">
```

```
  <figcaption>
    Figure 1: The basic structure of an HTML5 document
  </figcaption>
</figure>
```

3.4 Audio — <audio>

The `<audio>` element embeds audio without any plugin. Always include the `controls` attribute — without it, the player is invisible.

```
multimedia.html — or CodePen
<!-- In VSCode: local file via Live Server -->
<audio controls>
  <source src="audio/podcast.mp3" type="audio/mpeg">
  <source src="audio/podcast.ogg" type="audio/ogg">
  Your browser does not support HTML audio.
</audio>

<!-- In CodePen: public URL -->
<audio controls>
  <source src="https://example.com/audio/sample.mp3" type="audio/mpeg">
  Your browser does not support HTML audio.
</audio>
```

Attribute	Example	Effect
`controls`	controls	Shows the play/pause/volume/seek bar — always use this
`autoplay`	autoplay	Starts playing immediately — requires muted, often blocked by browsers
`loop`	loop	Repeats the audio when it ends
`muted`	muted	Starts with audio muted
`preload`	preload="auto"	auto: load everything. metadata: load only duration. none: load nothing

Why Multiple <source> Elements?

Different browsers support different audio formats. By listing multiple sources, the browser picks the first one it can play:

```
multimedia.html
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">  <!-- Chrome, Firefox,
Safari, Edge -->
  <source src="audio.ogg" type="audio/ogg">   <!-- Firefox fallback -->
  <source src="audio.wav" type="audio/wav">   <!-- Last resort -->
```

```
  <p>Your browser cannot play this audio.
     <a href="audio.mp3">Download instead</a>.
  </p>
</audio>
```

Try in CodePen

1. Go to codepen.io/pen/new
2. In the HTML panel, paste an <audio> element with a public MP3 URL.
   Example free audio URL: https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3
3. Try adding the loop, muted, and autoplay attributes one at a time.
4. Notice how autoplay alone is blocked by the browser — add muted alongside it.

3.5 Video — <video>

The `<video>` element works exactly like `<audio>` but for video files. The extra `poster` attribute lets you set a thumbnail shown before the video plays.

```
multimedia.html — or CodePen
<!-- VSCode + Live Server -->
<video controls width="640" height="360" poster="images/thumb.jpg">
  <source src="video/lecture.mp4" type="video/mp4">
  <source src="video/lecture.webm" type="video/webm">
  Your browser does not support HTML video.
</video>

<!-- CodePen: public video URL -->
<video controls width="640" poster="https://example.com/thumb.jpg">
  <source src="https://example.com/video/sample.mp4" type="video/mp4">
</video>
```

Attribute	Example	Effect
`controls`	controls	Shows playback controls — always include
`width`	width="640"	Width of the video player in pixels
`height`	height="360"	Height of the video player in pixels
`poster`	poster="thumb.jpg"	Thumbnail image shown before video starts playing
`autoplay`	autoplay	Plays immediately — must pair with muted
`loop`	loop	Replays the video when it ends
`muted`	muted	Starts muted — required for autoplay
`playsinline`	playsinline	Plays inline on iOS (prevents forced full-screen)

Attribute	Example	Effect
`preload`	preload="metadata"	Controls how much is pre-loaded before user presses play

Recommended Video Formats

- **MP4 (H.264)** — most widely supported; use as your first <source>
- **WebM (VP9)** — open format; great quality; Chrome, Firefox, Edge
- **OGV** — older format; rarely needed today

Try in CodePen

1. Open a new pen on codepen.io
2. Paste this into the HTML panel:
   <video controls width="500" poster="https://via.placeholder.com/500x280">
   <source src="https://www.w3schools.com/html/mov_bbb.mp4" type="video/mp4">
   </video>
3. Add poster, loop, and muted attributes and see how each changes the behaviour.
4. Try removing controls — notice the video becomes uncontrollable.

3.6 Embedding External Videos — <iframe>

For YouTube, Vimeo, or other hosted video platforms, use `<iframe>` instead of `<video>`. These platforms handle the streaming and format conversion for you.

```
CodePen HTML panel
<!-- YouTube embed — copy the embed code from YouTube: -->
<!-- Share → Embed → Copy the iframe code -->
<iframe
  width="560"
  height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="YouTube video player"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media;
gyroscope"
  allowfullscreen>
</iframe>
```

How to Get a YouTube Embed Code

1. Open any YouTube video
2. Click Share (below the video)

3. Click Embed
4. Copy the entire <iframe>...</iframe> code
5. Paste it directly into your CodePen HTML panel — it will work immediately

Try in CodePen — YouTube + Spotify Embeds

YouTube:
1. Go to any YouTube video → Share → Embed → Copy iframe
2. Paste into the CodePen HTML panel

Spotify (bonus):
1. Open any Spotify song/playlist in a browser
2. Click the '...' menu → Share → Embed track
3. Copy and paste the iframe into CodePen

Notice: These embeds ONLY work because CodePen serves from a real server.
They will NOT work if you open an HTML file directly from your Desktop (file://).

When to Use <video> vs <iframe>

<video> — use when you own and host the video file (on your server or CDN)
<iframe> — use when embedding from YouTube, Vimeo, or any external platform
Rule of thumb: If you have the file → <video>. If you have a URL to a platform → <iframe>.

3.7 Figure + Figcaption with Video

```
multimedia.html
<figure>
  <video controls width="600" poster="images/thumb.jpg">
    <source src="video/intro.mp4" type="video/mp4">
    <source src="video/intro.webm" type="video/webm">
  </video>
  <figcaption>Video 1: Introduction to HTML — Module 2</figcaption>
</figure>
```

3.8 Multimedia Best Practices

5. **Always add alt text to images** — never skip it
6. **Always include controls** on <audio> and <video>
7. **Provide multiple <source> elements** for audio and video (MP4 + WebM, MP3 + OGG)
8. **Include fallback text** inside <audio> and <video> for old browsers

9. **Use loading='lazy'** on all images below the fold
10. **Never use autoplay without muted** — browsers block it
11. **Use <figure> and <figcaption>** to pair media with its caption semantically
12. **Optimise file sizes** — use WebP for images, compress videos before uploading
13. **Use CodePen for embeds and external URLs** — VSCode Live Server for local files

3.9 Complete Multimedia Page Example

This example works best in VSCode with Live Server for local files, and in CodePen for the YouTube embed:

```
multimedia.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Media Gallery</title>
  <style>
    img, video { max-width: 100%; height: auto; }
    figure { margin: 0 0 2rem 0; }
    figcaption { color: #555; font-style: italic; margin-top: 0.5rem; }
  </style>
</head>
<body>
  <h1>TS Academy — Media Gallery</h1>

  <!-- Image -->
  <figure>
    <img src="images/campus.jpg"
         alt="TS Academy campus building in Lagos"
         loading="lazy">
    <figcaption>Our campus in Lagos</figcaption>
  </figure>

  <!-- Audio -->
  <h2>Listen to the Lecture</h2>
  <audio controls>
    <source src="audio/lecture-1.mp3" type="audio/mpeg">
    <source src="audio/lecture-1.ogg" type="audio/ogg">
    <p>Audio not supported. <a href='audio/lecture-1.mp3'>Download</a></p>
  </audio>

  <!-- Video -->
  <figure>
    <video controls width="640" poster="images/demo-thumb.jpg">
      <source src="video/demo.mp4" type="video/mp4">
      <source src="video/demo.webm" type="video/webm">
      <p>Video not supported. <a href='video/demo.mp4'>Download</a></p>
    </video>
```

```
      <figcaption>Demo: Building your first webpage</figcaption>
    </figure>

    <!-- YouTube embed (use in CodePen or VSCode Live Server) -->
    <h2>Bonus: YouTube Lesson</h2>
    <iframe width="560" height="315"
      src="https://www.youtube.com/embed/VIDEO_ID"
      title="HTML Tutorial"
      frameborder="0"
      allowfullscreen>
    </iframe>
  </body>
  </html>
```

3.10 Practice Task — Multimedia

Task: Media Gallery Page (CodePen + VSCode)

PART A — In VSCode (Live Server required):

1. Create multimedia.html
2. Add at least 3 <img> tags with alt text — use images saved in an 'images/' folder
3. Add one <audio> player pointing to a local .mp3 file
4. Add one <video> player with a poster image
5. Wrap all media in <figure> and <figcaption>

PART B — In CodePen:

1. Open codepen.io/pen/new
2. Embed a YouTube video of your choice (Share → Embed)
3. Add an <audio> using this public URL:
   https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3
4. Add an <img> using a public image URL from unsplash.com
5. Add a Spotify embed (any song — Share → Embed track)
6. Experiment with adding loop, muted, and autoplay to your audio/video

∑ Quick Reference Summary

Tags at a Glance

Section 1 — HTML Tables

Tag / Attribute	Use It For
`<table>`	Outer container for the entire table
`<tr>`	One horizontal row
`<th>`	Header cell — bold, centred, use scope='col/row'
`<td>`	Data cell — holds your content
`<thead>`	Groups header rows
`<tbody>`	Groups body data rows
`<tfoot>`	Groups footer / summary rows
`<caption>`	Title for the table — first child of <table>
`colspan="n"`	Merges cell across n columns
`rowspan="n"`	Merges cell down n rows

Section 2 — HTML Forms

Tag / Attribute	Use It For
`<form>`	Container — set action and method
`<input>`	Main input control — use type attribute to change behaviour
`<label>`	Labels an input — for='id' must match input's id
`<textarea>`	Multi-line text input
`<select>`	Dropdown list
`<option>`	One item inside a <select>
`<optgroup>`	Groups related <option> items in a dropdown
`<fieldset>`	Visually and semantically groups related inputs
`<legend>`	Title for a <fieldset>
`<button>`	Clickable button: submit / reset / button
`required`	HTML5 validation — field must be filled
`placeholder`	Ghost hint text shown before user types
`name`	Key sent to the server — every input needs this
`id`	Links input to its <label> — every input needs this

Section 3 — HTML Multimedia

Tag / Attribute	Use It For
`<img>`	Embed an image — src and alt are required
`<audio>`	Audio player — always include controls
`<video>`	Video player — always include controls
`<source>`	Specifies one media source inside <audio> or <video>
`<iframe>`	Embeds external content: YouTube, Spotify, maps, etc.
`<figure>`	Semantic wrapper for media
`<figcaption>`	Caption for a <figure>
`alt="…"`	Describes an image — required for accessibility and SEO
`poster="…"`	Thumbnail shown before a video plays
`loading="lazy"`	Defers image loading — improves performance
`controls`	Shows playback controls on audio/video — always use
`loop`	Repeats media when it ends
`muted`	Starts muted — required for autoplay to work

Further Resources

MDN Web Docs (best reference): https://developer.mozilla.org/en-US/docs/Web/HTML
CodePen (practice embeds): https://codepen.io
VSCode Download: https://code.visualstudio.com
Free test images: https://unsplash.com | https://picsum.photos
Free test audio: https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3
Free test video: https://www.w3schools.com/html/mov_bbb.mp4